GoQuant

Assignment

Objective:- Develop a high-performance cryptocurrency matching engine. This engine should implement core trading functionalities based on REG NMS-inspired principles of price-time priority and internal order protection. Additionally, the engine must generate its own stream of trade execution data.

Executive Summary:- A lightweight, real-time matching engine for limit/market orders with WebSocket push. Core goals:

- Deterministic, fair matching (price–time priority) with IOC/FOK handling.
- Low latency (<5ms p50 for match on a single node) and high throughput via in-memory order book + append-only log.
- Observable and operable: metrics, structured logs, clear runbook & tests.
- Horizontally scalable: partition by symbol, stateless API tier, shared persistence**.**

Architecture & Diagram :-

Components

- API Gateway: REST for CRUD & snapshots, WebSocket (WS) for market data streams.
- Match Engine(s): per-symbol workers holding in-memory books; read/write via command bus; emit trades & book deltas.
- Event Log: append-only (Kafka/Redpanda or Postgres WAL table) for durability & replay.
- Persistence: Postgres for orders, trades, snapshots; Redis for idempotency, rate limits, and pub/sub fan-out.
- Front-end: React SPA consuming REST + WS.

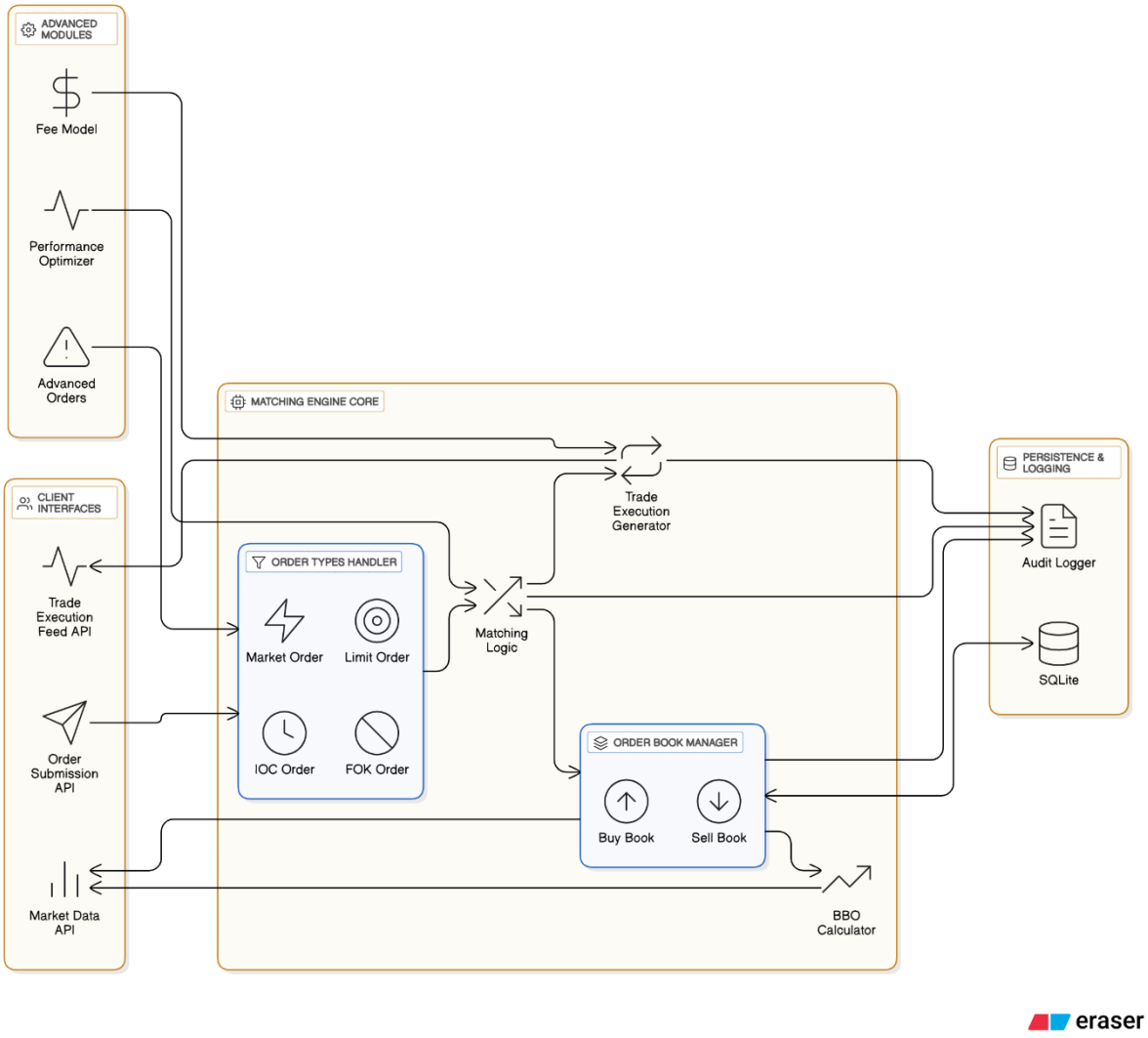

Fig. Architecture Diagram

Data Flow

1. Client places order → API validates & enqueues a command.
2. Engine consumes commands FIFO per symbol, mutates book, emits events (trade, partial fill, cancel).
3. Events persisted (log + DB) and fanned out to WS subscribers. Periodic snapshots enable fast recovery.

Tech Stack

Backend: Django

- Django + Django Channels
    - Pros: batteries-included ORM/migrations/admin; robust auth; Channels integrates WS; good for ops velocity.
    - Cons: Slightly heavier; ASGI performance is solid but not the fastest.
- FastAPI + Uvicorn
    - Pros: very fast, type-driven, minimal; excellent for microservices.
    - Cons: Must assemble pieces (auth/admin/migrations) .

Frontend : React

Matching Engine Design

Core invariants

- Price–time priority for limit orders within each side.
- Buy book sorted by **descending** price, FIFO within price level.
- Sell book sorted by **ascending** price, FIFO within level.
- An order's remaining quantity ≥ 0; filled_qty + remaining = original_qty.
- Trades are symmetric: sum(buy_fills) == sum(sell_fills) per trade id.

Data structures

```
# Python types (engine process)
from collections import deque
from dataclasses import dataclass, field

@dataclass
class Order:
id: int
symbol: str
side: str # "buy" | "sell"
type: str # "limit" | "market" | "ioc" | "fok"
price: int # price in ticks (e.g., paise)
qty: int
ts: int # monotonic sequence per symbol
```

```
@dataclass
class PriceLevel:
price: int
queue: deque[Order] = field(default_factory=deque)

class OrderBook:
bids: dict[int, PriceLevel] # price -> level
asks: dict[int, PriceLevel]
bid_prices: list[int] # max-heap (or sorted list desc)
ask_prices: list[int] # min-heap (or sorted list asc)
```

Matching algorithm (limit buy example)

1. While best_ask.price ≤ buy.price and qty > 0: pop FIFO from best level, execute min(qty, level_head.qty).
2. Emit Trade events and update remaining.
3. If order remains and type in {limit} → enqueue to bid price level; if IOC → cancel remains; if FOK and not fully matched → rollback fills & cancel.

Complexities

- Insert/remove: O(log P) for heap + O(1) FIFO ops; P = number of price levels.
- Match loop depends on crossed volume; per trade O(1) bookkeeping.

Idempotency

- Deduplicate commands by client_order_id via Redis SET/EX or Postgres ON CONFLICT DO NOTHING.

Recovery

- On start: load latest snapshot (per symbol) + replay events from log offset.

5. APIs

REST (JSON)

- POST /api/orders
    - Body: { symbol, side, type, qty, price?, client_id? }
    - Returns: { order_id, status: "accepted"|"rejected", reason? }
- GET /api/snapshot?symbol=BTC-USD&depth=50
    - Returns top N bids/asks with aggregated sizes and last_trade.

- **DELETE /api/orders/{id} (cancel)**
- **GET /api/orders/{id} (status)**

Example snapshot response

{

"symbol": "ABC",

"bids": [[1199, 8], [1198, 5]],

"asks": [[1201, 7], [1202, 9]],

"last_trade": {"px": 1200, "qty": 2, "ts": 1710000001}

}

WebSocket Topics

- md.{symbol}.trades → {trade_id, buy_id, sell_id, px, qty, ts}
- md.{symbol}.l2 → {bids:[[px,qty],...], asks:[[px,qty],...], ts} deltas or snapshots
- orders.{user_id} → private execution reports: NEW, PARTIAL, FILLED, CANCELED with remaining qty

Trade-offs & Decision Log

Framework: Django + Channels vs FastAPI

- **Chosen**: Django + Channels
- **Why**: Faster product velocity (ORM, admin, auth); WS built-in via Channels; broad team familiarity.
- **Trade-off**: Slightly higher baseline latency vs minimal frameworks.
- **Risks**: Hot path execution inside web workers by mistake.
- **Revisit when**: API p95 > 70ms under load or admin isn't used → consider carving out a FastAPI gateway.

Engine Isolation: Separate process vs in-process

- Chosen: Separate engine workers per symbol communicating over Redis/Kafka.
- Why: Isolation of the latency-critical loop; language flexibility later (Rust/Go); easier horizontal scale.
- Trade-off: Cross-process hop adds ~0.3–1.5ms.

- Risks: At-least-once semantics → need idempotency.
- Revisit when: Single-host, ultra-low latency requirement where in-proc offers a big win.

Price/Qty Representation: Integers (ticks) vs floats

- **Chosen**: Integers
- **Why**: Determinism; avoids rounding drift; faster comparisons.
- **Trade-off**: Client must convert to tick size.
- **Revisit when**: Instruments require fractional tick grids that don't fit integer scaling.

Market Data Fan-out: WS push with batched deltas (50–100ms)

- **Why**: Cuts network chatter while keeping UX real-time.
- **Trade-off**: Slight staleness vs tick-for-tick streaming.
- **Revisit when**: Power users require depth-by-event streams → add a "raw" channel.

Matching Policy: Price-time vs Pro-rata / size-time

- **Chosen**: Price-time (FIFO within level)
- **Why**: Market norm, fairness, easier mental model.
- **Trade-off**: Large orders may wait behind many small ones.
- **Revisit when**: Venue incentives or liquidity programs demand alternative priority.